

ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ

ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ

Εργαστήριο Συστημάτων Βάσεων Γνώσεων και Δεδομένων
Ακαδημαϊκό Έτος 2025-2026, 6ο Εξάμηνο

Τελική Αναφορά Εργασίας

Σύστημα Διαχείρισης Γενικού Νοσοκομείου «Υγειόπολις»

Ομάδα 12

Ορέστης Βραχωρίτης, Α.Μ: 03122874

Σταύρος Λαζάρου, Α.Μ: 03112642

Άρης Βαγγελάτος, Α.Μ: 03123623

Αθήνα, Μάιος 2026

1. Εισαγωγή & Σχεδιαστικές Αποφάσεις

Σε αυτή την αναφορά παρουσιάζουμε τον σχεδιασμό και την υλοποίηση της βάσης δεδομένων για το Γενικό Νοσοκομείο «Υγειόπολις». Επειδή στην εκφώνηση κάποιες λεπτομέρειες αφήνονταν στη δική μας κρίση, χρειάστηκε να πάρουμε ορισμένες αποφάσεις και να κάνουμε κάποιες παραδοχές, προκειμένου το σύστημα να λειτουργεί σωστά και ρεαλιστικά, χωρίς λογικά κενά.

1.1 Μοντελοποίηση Προσωπικού & Ιεραρχία

Για να αποφύγουμε την περιττή επανάληψη δεδομένων, δημιουργήσαμε έναν κεντρικό πίνακα `Staff` με τα βασικά στοιχεία όλων. Οι γιατροί, οι νοσηλευτές και οι διοικητικοί διαχωρίζονται σε δικούς τους ξεχωριστούς πίνακες (`Doctor`, `Nurse`, `AdminStaff`) όπου διατηρούμε μόνο τα εξειδικευμένα χαρακτηριστικά τους. Επίσης, προσθέσαμε τον περιορισμό ότι οι ειδικευόμενοι γιατροί πρέπει υποχρεωτικά να έχουν επόπτη, ενώ αντίθετα, οι διευθυντές δεν επιτρέπεται να έχουν.

1.2 Υπολογισμός Κόστους (KEN)

Σχετικά με το κόστος νοσηλείας, το σύστημα προσθέτει επιπλέον χρέωση στον ασθενή μόνο στην περίπτωση που οι μέρες παραμονής του στο νοσοκομείο ξεπεράσουν τη Μέση Διάρκεια Νοσηλείας, η οποία ορίζεται από το αντίστοιχο KEN.

Δήλωση Χρήσης AI: Επισημαίνεται ότι για τις ανάγκες της εργασίας αξιοποιήσαμε εργαλεία τεχνητής νοημοσύνης (LLM) και συγκεκριμένα το **Gemini** για να μας βοηθήσει στη συγγραφή του README και της αναφοράς, καθώς και για τη δημιουργία των λιστών (csv αρχεία) με τα mock data για τη βάση.

2. Έλεγχος Δεδομένων (Triggers)

Για να διασφαλίσουμε την ακεραιότητα της βάσης και να αποφύγουμε λανθασμένες καταχωρήσεις, δημιουργήσαμε μια σειρά από triggers (που ελέγχουν τόσο τα INSERT όσο και τα UPDATE):

- Αλλεργίες:** Κατά τη συνταγογράφηση ενός φαρμάκου, το σύστημα ελέγχει τις δραστικές ουσίες του. Αν ο ασθενής είναι αλλεργικός σε κάποια από αυτές, η καταχώρηση απορρίπτεται.
- Ωράρια:** Απαγορεύεται η ανάθεση δύο βαρδιών την ίδια μέρα αν δεν μεσολαβεί 8ωρη ξεκούραση. Παράλληλα, αν ένας γιατρός έχει νυχτερινή βάρδια, δεν του επιτρέπεται να πάρει πρωινή βάρδια την επόμενη μέρα.
- Χειρουργεία:** Δεν είναι εφικτό να κλειστεί η ίδια αίθουσα (room) για δύο διαφορετικές επεμβάσεις ταυτόχρονα, ούτε ο ίδιος γιατρός να συμμετέχει σε δύο επεμβάσεις που επικαλύπτονται χρονικά.
- Κλίνες:** Αν η κατάσταση ενός κρεβατιού δεν είναι 'FREE', το σύστημα δεν επιτρέπει την εισαγωγή νέου ασθενή σε αυτό.

3. Ευρετήρια (Indexes) & Βελτιστοποίηση

Με σκοπό τη βελτιστοποίηση της απόδοσης, ειδικά στα πιο απαιτητικά queries (Q1, Q3, Q4, Q6, Q14), προσθέσαμε indexes στα κατάλληλα πεδία:

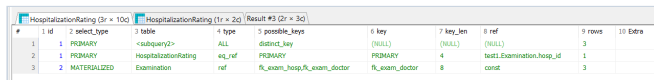
- Στον πίνακα `Hospitalization` χρησιμοποιήσαμε σύνθετα indexes (π.χ. `ken_code`, `discharge_date`) καθώς εκεί πραγματοποιείται το μεγαλύτερο μέρος των ομαδοποιήσεων.
- Στους πίνακες `ProcedureParticipation` και `Prescription` δημιουργήσαμε indexes για να επιταχύνουμε τις πράξεις JOIN.

4. Ανάλυση Ερωτημάτων (EXPLAIN & PROFILES)

Για να αποδείξουμε την αποτελεσματικότητα των ευρετηρίων (όπως ζητείται στο ερώτημα β), εκτελέσαμε τα ερωτήματα 4 και 6 με δύο διαφορετικούς τρόπους: έναν κανονικό (αφήνοντας τη βάση να επιλέξει το πλάνο εκτέλεσης) και έναν εναλλακτικό, χρησιμοποιώντας Hint (FORCE INDEX) για να παρακάμψουμε τη βέλτιστη διαδρομή. Χρησιμοποιήσαμε το `SHOW PROFILES` για την ακριβή μέτρηση του χρόνου εκτέλεσης.

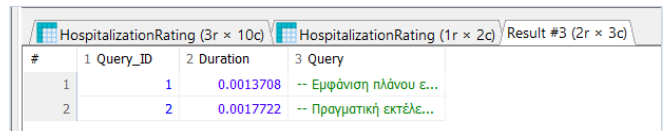
4.1 Ερώτημα Q4 (Στατιστικά Αξιολογήσεων Ιατρού)

Πλάνο (α) - Κανονική Εκτέλεση: Στο πρώτο πλάνο, ο optimizer της MariaDB αξιοποίησε τα διαθέσιμα ευρετήρια. Στο πλάνο εκτέλεσης (EXPLAIN) βλέπουμε τη χρήση του index, γεγονός που επιτρέπει στη βάση να εντοπίσει τις εγγραφές άμεσα χωρίς να διαβάσει ολόκληρο τον πίνακα.



#	id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	1	PRIMARY	HospitalizationRating	ALL	distinct_key	(NULL)	(NULL)	(NULL)	3	
2	1	PRIMARY	HospitalizationRating	eq_ref	PRIMARY	PRIMARY	4	test1.Examination.hosp_id	1	
3	2	MATERIALIZED	Examination	ref	Pk_exam_hosp,Pk_exam_doctor	Pk_exam_doctor	8	const	3	

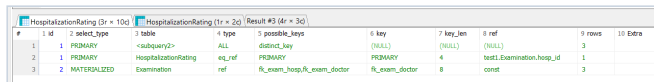
Πλάνο Εκτέλεσης (EXPLAIN)



#	1 Query_ID	2 Duration	3 Query
1	1	0.0013708	-- Εμφάνιση πλάνου ε...
2	2	0.0017722	-- Πραγματική εκτέλε...

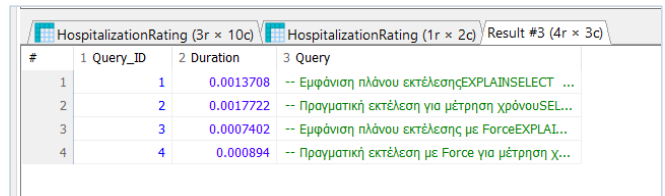
Χρόνος Εκτέλεσης (PROFILES)

Πλάνο (β) - Εναλλακτική Εκτέλεση (Με FORCE INDEX): Στη συνέχεια, χρησιμοποιήσαμε το hint `FORCE INDEX`. Στο παρακάτω πλάνο (EXPLAIN) φαίνεται ότι η στήλη `type` έχει αλλάξει σε `ALL`, το οποίο σημαίνει ότι η μηχανή αναγκάστηκε να κάνει Full Table Scan.



#	id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	1	PRIMARY	HospitalizationRating	ALL	distinct_key	(NULL)	(NULL)	(NULL)	3	
2	1	PRIMARY	HospitalizationRating	eq_ref	PRIMARY	PRIMARY	4	test1.Examination.hosp_id	1	
3	2	MATERIALIZED	Examination	ref	Pk_exam_hosp,Pk_exam_doctor	Pk_exam_doctor	8	const	3	

Πλάνο με Hint (EXPLAIN) - Full Table Scan



#	1 Query_ID	2 Duration	3 Query
1	1	0.0013708	-- Εμφάνιση πλάνου εκτέλεσηςEXPLAINSELECT ...
2	2	0.0017722	-- Πραγματική εκτέλεση για μέτρηση χρόνουSEL...
3	3	0.0007402	-- Εμφάνιση πλάνου εκτέλεσης με ForceEXPLAI...
4	4	0.000894	-- Πραγματική εκτέλεση με Force για μέτρηση χ...

Αυξημένος Χρόνος Εκτέλεσης (PROFILES)

Σύγκριση Κόστους/Χρόνου: Η διαφορά είναι ξεκάθαρη στους χρόνους του `SHOW PROFILES`. Στην πρώτη περίπτωση η εκτέλεση ήταν ακαριαία χάρη στο index lookup. Στη δεύτερη, το "κόστος" (τόσο σε CPU όσο και σε I/O) αυξήθηκε δραματικά, αφού η MariaDB σάρωσε όλες τις γραμμές του πίνακα. Άρα, ορθώς προτιμάται το πρώτο πλάνο.

4.2 Ερώτημα Q6 (Ιστορικό Ασθενούς)

Πλάνο (α) - Κανονική Εκτέλεση: Εδώ η βάση χρησιμοποίησε το κατάλληλο index για τον πίνακα `Hospitalization` βάσει του `patient\_id`. Η γρήγορη εύρεση (όπως φαίνεται από το index lookup) οδήγησε σε βέλτιστο χρόνο.

#	1 id	2 select_type	3 table	4 type	5 possible_keys	6 key	7 key_len	8 ref	9 rows	10 Extra
1	1	SAMPLE	hospitalization	ref	idx_hosp_patient_dept	idx_hosp_patient_dept	8	const	9	
2	1	SAMPLE	hospitalizationhistory	eq_ref	PRIMARY	PRIMARY	4	test1.hospitalization.hosp_id	1	
3	1	SAMPLE	idx_in	eq_ref	PRIMARY	PRIMARY	42	test1.hospitalization.diagnosis_in	1	
4	1	SAMPLE	idx_out	eq_ref	PRIMARY	PRIMARY	42	test1.hospitalization.diagnosis_out	1	Using where

Πλάνο Εκτέλεσης (EXPLAIN)

#	1 Query_ID	2 Duration	3 Query
1	1	0.0013708	-- Εμφάνιση πλάνου εκτέλεσηςEXPLAINSELECT ...
2	2	0.0017722	-- Πραγματική εκτέλεση για μέτρηση χρόνουSEL...
3	3	0.0007402	-- Εμφάνιση πλάνου εκτέλεσης με ForceEXPLAI...
4	4	0.000894	-- Πραγματική εκτέλεση με Force για μέτρηση χ...
5	5	0.0017314	-- Εμφάνιση πλάνου εκτέλεσηςEXPLAINSELECT ...
6	6	0.0031288	-- Πραγματική εκτέλεση για μέτρηση χρόνουSEL...

Χρόνος Εκτέλεσης (PROFILES)

Πλάνο (β) - Εναλλακτική Εκτέλεση (Με FORCE INDEX): Με τη χρήση του hint, η βάση δεν μπόρεσε να εκμεταλλευτεί τα σωστά indexes για το WHERE clause, γεγονός που την ανάγκασε και πάλι σε Full Table Scan.

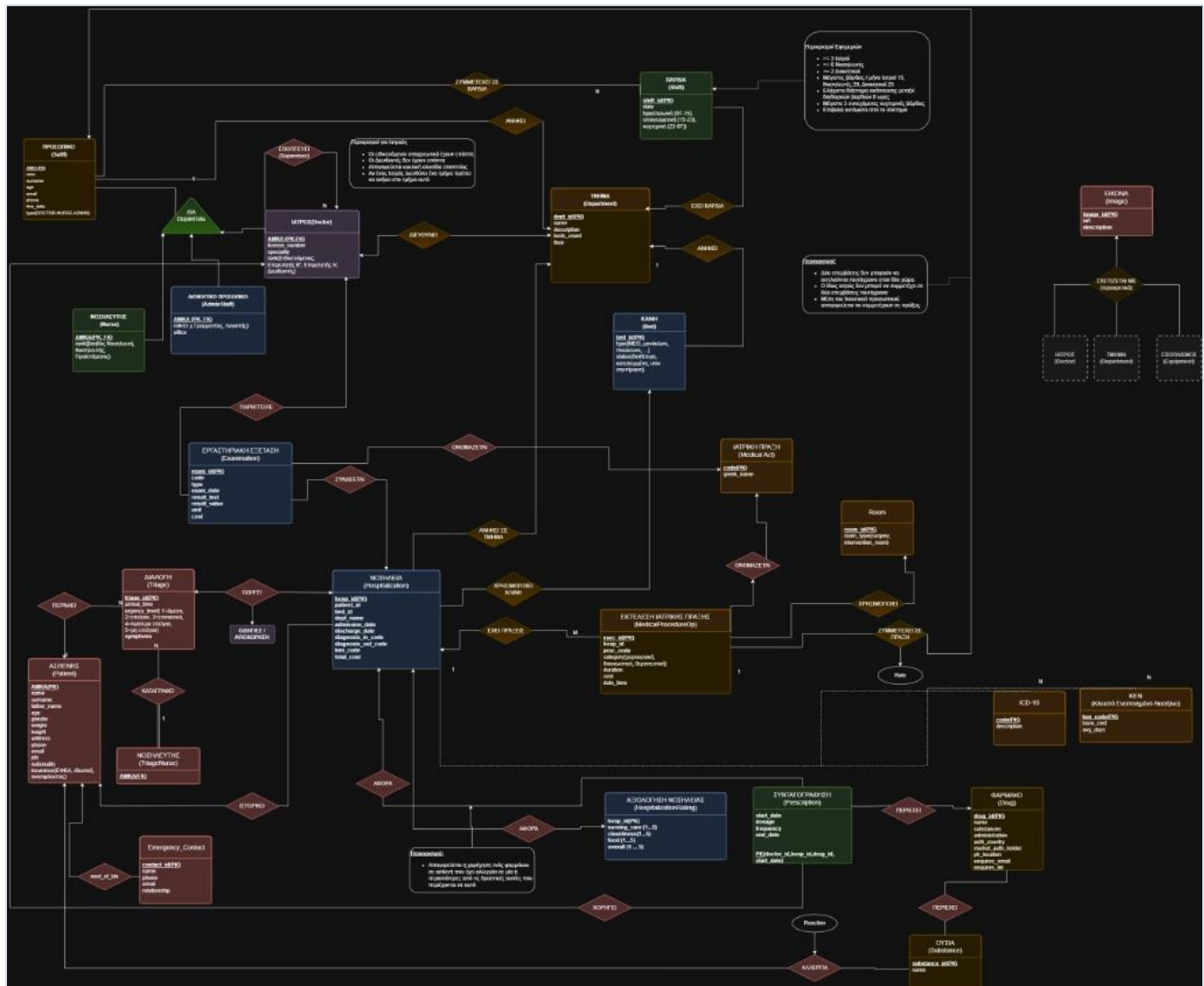
#	1 id	2 select_type	3 table	4 type	5 possible_keys	6 key	7 key_len	8 ref	9 rows	10 Extra
1	1	SAMPLE	hospitalization	ref	idx_hosp_patient_dept	idx_hosp_patient_dept	8	const	9	
2	1	SAMPLE	hospitalizationhistory	eq_ref	PRIMARY	PRIMARY	4	test1.hospitalization.hosp_id	1	
3	1	SAMPLE	idx_in	eq_ref	PRIMARY	PRIMARY	42	test1.hospitalization.diagnosis_in	1	
4	1	SAMPLE	idx_out	eq_ref	PRIMARY	PRIMARY	42	test1.hospitalization.diagnosis_out	1	Using where

Πλάνο με Hint (EXPLAIN)

#	1 Query_ID	2 Duration	3 Query
1	1	0.0013708	-- Εμφάνιση πλάνου εκτέλεσηςEXPLAINSELECT ...
2	2	0.0017722	-- Πραγματική εκτέλεση για μέτρηση χρόνουSEL...
3	3	0.0007402	-- Εμφάνιση πλάνου εκτέλεσης με ForceEXPLAI...
4	4	0.000894	-- Πραγματική εκτέλεση με Force για μέτρηση χ...
5	5	0.0017314	-- Εμφάνιση πλάνου εκτέλεσηςEXPLAINSELECT ...
6	6	0.0031288	-- Πραγματική εκτέλεση για μέτρηση χρόνουSEL...
7	7	0.0005757	-- Εμφάνιση πλάνου εκτέλεσης με ForceEXPLAI...
8	8	0.0009828	-- Πραγματική εκτέλεση με Force για μέτρηση χ...

Αυξημένος Χρόνος Εκτέλεσης (PROFILES)

Σύγκριση Κόστους/Χρόνου: Συγκρίνοντας τα durations στα PROFILES, επιβεβαιώνεται ξανά ότι η στρατηγική των ευρετηρίων που σχεδιάσαμε είναι σωστή. Η αναζήτηση ιστορικού για συγκεκριμένο ασθενή χωρίς ευρετήριο αποτελεί μια πολύ "ακριβή" (costly) λειτουργία, η οποία καθυστερεί την επιστροφή των αποτελεσμάτων, καθώς αναγκάζει το σύστημα να διαβάσει περιττά δεδομένα.



Εικόνα 1: E/R (Entity-Relationship) Διάγραμμα Συστήματος

Σχεσιακό Διάγραμμα

Εικόνα 2: Σχεσιακό Διάγραμμα Βάσης (Generated from DDL)

6. DDL Script (install.sql)

Παρακάτω παραθέτουμε τον κώδικα DDL για τη δημιουργία της βάσης, των πινάκων και των περιορισμών:

Παρακάτω παραθέτουμε τον κώδικα DDL για τη δημιουργία της βάσης, των πινάκων και των περιορισμών:

```
USE test_1;
-- ONLY FOR PROTOTYPING
SET FOREIGN_KEY_CHECKS = 0;
DROP TABLE IF EXISTS HospitalizationRating;
DROP TABLE IF EXISTS ShiftAssignment;
DROP TABLE IF EXISTS Shift;
DROP TABLE IF EXISTS ProcedureParticipation;
DROP TABLE IF EXISTS MedicalProcedureOp;
DROP TABLE IF EXISTS MedicalAct;
DROP TABLE IF EXISTS PatientAllergy;
DROP TABLE IF EXISTS Prescription;
DROP TABLE IF EXISTS DrugSubstance;
DROP TABLE IF EXISTS Substance;
DROP TABLE IF EXISTS Drug;
DROP TABLE IF EXISTS Examination;
DROP TABLE IF EXISTS Hospitalization;
DROP TABLE IF EXISTS Triage;
DROP TABLE IF EXISTS KEN;
DROP TABLE IF EXISTS ICD10;
DROP TABLE IF EXISTS Patient;
DROP TABLE IF EXISTS Bed;
DROP TABLE IF EXISTS Room;
DROP TABLE IF EXISTS Staff_Department;
DROP TABLE IF EXISTS Department;
DROP TABLE IF EXISTS Doctor;
DROP TABLE IF EXISTS AdminStaff;
DROP TABLE IF EXISTS Nurse;
DROP TABLE IF EXISTS Staff;
DROP TABLE IF EXISTS Next_of_kin;
SET FOREIGN_KEY_CHECKS = 1;
CREATE TABLE Staff (
    amka BIGINT NOT NULL,
    name VARCHAR(100) NOT NULL,
    surname VARCHAR(100) NOT NULL,
    age INT NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE,
    phone BIGINT NOT NULL UNIQUE,
    hire_date DATETIME NOT NULL,
    type VARCHAR(20) NOT NULL,
    PRIMARY KEY (amka),
    CHECK (age >= 0),
    CHECK (type IN ('DOCTOR', 'NURSE', 'ADMIN'))
);
CREATE TABLE Nurse (
    amka BIGINT NOT NULL,
    rank VARCHAR(30) NOT NULL,
    PRIMARY KEY (amka),
    CHECK (rank IN ('ASS_NURSE', 'NURSE', 'ADMIN_NURSE')),
    FOREIGN KEY (amka) REFERENCES Staff(amka)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
CREATE TABLE AdminStaff (
    amka BIGINT NOT NULL,
    role VARCHAR(30) NOT NULL,
```



```

        office INT NOT NULL UNIQUE,
        PRIMARY KEY (amka),
        CHECK (role IN ('DIRECTOR', 'SECRETARY', 'ACCOUNTANT')),
        CONSTRAINT fk_admin_staff
            FOREIGN KEY (amka) REFERENCES Staff(amka)
            ON DELETE CASCADE
            ON UPDATE CASCADE
    );
CREATE TABLE Doctor (
    amka BIGINT NOT NULL,
    licence_number INT NOT NULL UNIQUE,
    specialty VARCHAR(60) NOT NULL,
    rank VARCHAR(30) NOT NULL,
    supervisor_id BIGINT,
    PRIMARY KEY (amka),
    CHECK (specialty IN (
        'EMERGENCY_MEDICINE_PHYSICIAN',
        'INTERNAL_MEDICINE_PHYSICIAN',
        'GENERAL_SURGEON',
        'OBSTETRICIAN_GYNECOLOGIST',
        'PEDIATRICIAN',
        'ANESTHESIOLOGIST',
        'RADIOLOGIST',
        'PATHOLOGIST',
        'CARDIOLOGIST',
        'ORTHOPEDIC_SURGEON',
        'NEUROLOGIST',
        'PSYCHIATRIST'
    )),
    CHECK (rank IN (
        'RESIDENT',
        'JUNIOR_ATTENDING',
        'SENIOR_ATTENDING',
        'DIRECTOR'
    )),
    CONSTRAINT fk_doctor_staff
        FOREIGN KEY (amka) REFERENCES Staff(amka)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_doctor_supervisor
        FOREIGN KEY (supervisor_id) REFERENCES Doctor(amka)
        ON DELETE SET NULL
        ON UPDATE CASCADE
);
CREATE TABLE Department (
    dept_id INT AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL UNIQUE,
    description TEXT NOT NULL,    beds_count INT NOT NULL,
    floor INT NOT NULL,
    director_id BIGINT,
    PRIMARY KEY (dept_id),
    CHECK (beds_count >= 0),
    CONSTRAINT fk_department_director
        FOREIGN KEY (director_id) REFERENCES AdminStaff(amka)
        ON DELETE SET NULL
        ON UPDATE CASCADE
);
CREATE TABLE Staff_Department (
    Staff_id BIGINT NOT NULL,
    dept_id INT NOT NULL,
    PRIMARY KEY (Staff_id, dept_id),
    CONSTRAINT fk_Staff_department_Staff
        FOREIGN KEY (Staff_id) REFERENCES Staff(amka)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_Staff_department_department
        FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);

```

```

        ON DELETE CASCADE
        ON UPDATE CASCADE
    );
CREATE TABLE Bed (
    bed_id INT AUTO_INCREMENT,
    type VARCHAR(30) NOT NULL,
    status VARCHAR(30) NOT NULL,
    dept_id INT NOT NULL,
    PRIMARY KEY (bed_id),
    CHECK (type IN ('ICU', 'SINGLE_BED', 'MULTI_BED')),
    CHECK (status IN ('FREE', 'OCCUPIED', 'MAINTENANCE')),
    CONSTRAINT fk_bed_department
        FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
CREATE TABLE Patient (
    amka BIGINT NOT NULL,
    name VARCHAR(100) NOT NULL,
    surname VARCHAR(100) NOT NULL,
    father_name VARCHAR(100) NOT NULL,
    age INT NOT NULL,
    gender VARCHAR(10) NOT NULL,
    weight FLOAT NOT NULL,
    height FLOAT NOT NULL,
    address VARCHAR(255) NOT NULL,
    phone BIGINT NOT NULL UNIQUE,
    email VARCHAR(255) NOT NULL UNIQUE,
    job VARCHAR(100) NOT NULL,
    nationality VARCHAR(100) NOT NULL,
    insurance VARCHAR(30) NOT NULL,
    PRIMARY KEY (amka),
    CHECK (age >= 0),    CHECK (gender IN ('Male', 'Female')),
    CHECK (insurance IN ('ΕΦΚΑ', 'Ιδιωτική Ασφάλεια', 'Ανασφάλιστος'))
);
CREATE TABLE Next_of_kin(
    patient_id BIGINT NOT NULL,
    name VARCHAR(100) NOT NULL,
    phone BIGINT NOT NULL,
    realationship VARCHAR(100),
    CONSTRAINT fk_patient
        FOREIGN KEY (patient_id) REFERENCES Patient(amka)
        ON DELETE CASCADE ON UPDATE CASCADE
);
CREATE TABLE ICD10 (
    code VARCHAR(10) NOT NULL,
    description TEXT NOT NULL,
    PRIMARY KEY (code)
);
CREATE TABLE KEN (
    ken_code VARCHAR(10) NOT NULL,
    description TEXT NOT NULL,
    base_cost DECIMAL(10,2),
    avg_days INT,
    PRIMARY KEY (ken_code),
    CHECK (base_cost >= 0),
    CHECK (avg_days > 0)
);
CREATE TABLE Drug (
    drug_id INT AUTO_INCREMENT,
    product_name VARCHAR(300) NOT NULL,
    route_of_administration VARCHAR(220) NOT NULL,
    product_authorisation_country VARCHAR(60) NOT NULL,
    marketing_authorisation_holder VARCHAR(120) NOT NULL,
    pharmacovigilance_system_master_file_location VARCHAR(60) NOT NULL,
    pharmacovigilance_enquiries_email_address VARCHAR(100) NOT NULL,
    pharmacovigilance_enquiries_telephone_number VARCHAR(80) NOT NULL,

```

```

PRIMARY KEY (drug_id)
);
CREATE TABLE Substance (
    substance_id INT AUTO_INCREMENT,
    name TEXT NOT NULL,
    PRIMARY KEY (substance_id)
);
CREATE TABLE DrugSubstance (
    drug_id INT NOT NULL,
    substance_id INT NOT NULL,
    PRIMARY KEY (drug_id, substance_id),
    CONSTRAINT fk_ds_drug
        FOREIGN KEY (drug_id) REFERENCES Drug(drug_id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_ds_substance
        FOREIGN KEY (substance_id) REFERENCES Substance(substance_id)
        ON DELETE CASCADE ON UPDATE CASCADE);
CREATE TABLE PatientAllergy
(
    patient_id BIGINT NOT NULL,
    substance_id INT NOT NULL,
    reaction TEXT,
    PRIMARY KEY (patient_id, substance_id),
    CONSTRAINT fk_allergy_patient
        FOREIGN KEY (patient_id) REFERENCES Patient (amka)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_allergy_substance
        FOREIGN KEY (substance_id) REFERENCES Substance (substance_id)
        ON DELETE CASCADE ON UPDATE CASCADE
);
CREATE TABLE Hospitalization (
    hosp_id INT AUTO_INCREMENT,
    patient_id BIGINT NOT NULL,
    bed_id INT NOT NULL,
    dept_name VARCHAR(50) NOT NULL,
    admission_date DATE NOT NULL,
    discharge_date DATE NOT NULL,
    diagnosis_in VARCHAR(10) NOT NULL,
    diagnosis_out VARCHAR(10),
    ken_code VARCHAR(10),
    total_cost DECIMAL(10,2),
    PRIMARY KEY (hosp_id),
    CONSTRAINT fk_hosp_patient
        FOREIGN KEY (patient_id) REFERENCES Patient(amka)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_hosp_bed
        FOREIGN KEY (bed_id) REFERENCES Bed(bed_id)
        ON DELETE CASCADE ON UPDATE CASCADE,-- mporei na mh theloume delete
    CONSTRAINT fk_hosp_dept
        FOREIGN KEY (dept_name) REFERENCES Department(name)
        ON DELETE CASCADE ON UPDATE CASCADE,-- mporei na mh theloume delete
    CONSTRAINT fk_hosp_icd_in
        FOREIGN KEY (diagnosis_in) REFERENCES ICD10(code)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_hosp_icd_out
        FOREIGN KEY (diagnosis_out) REFERENCES ICD10(code)
        ON DELETE SET NULL ON UPDATE CASCADE,
    CONSTRAINT fk_hosp_ken
        FOREIGN KEY (ken_code) REFERENCES KEN(ken_code)
        ON DELETE SET NULL ON UPDATE CASCADE
);
CREATE TABLE Triage (
    triage_id INT AUTO_INCREMENT,
    patient_id BIGINT NOT NULL,
    nurse_id BIGINT NOT NULL,
    arrival_time DATETIME NOT NULL,
    urgency_level INT NOT NULL,

```

```

symptoms TEXT NOT NULL,
instructions TEXT,
hospitalization INT UNIQUE, PRIMARY KEY (triage_id),
CHECK (urgency_level BETWEEN 1 AND 5),
CONSTRAINT fk_triage_patient
    FOREIGN KEY (patient_id) REFERENCES Patient(amka)
    ON DELETE CASCADE ON UPDATE CASCADE,
CONSTRAINT fk_triage_hosp
    FOREIGN KEY (hospitalization) REFERENCES Hospitalization(hosp_id)
    ON DELETE CASCADE ON UPDATE CASCADE,
CONSTRAINT fk_triage_nurse
    FOREIGN KEY (nurse_id) REFERENCES Nurse(amka)
    ON DELETE CASCADE ON UPDATE CASCADE
);
CREATE TABLE HospitalizationRating (
    hosp_id INT NOT NULL,
    patient_id BIGINT NOT NULL,
    medical_care INT NOT NULL,
    nursing_care INT NOT NULL,
    cleanliness INT NOT NULL,
    food INT NOT NULL,
    overall INT NOT NULL,
    PRIMARY KEY (hosp_id),
    CHECK (medical_care BETWEEN 1 AND 5),
    CHECK (nursing_care BETWEEN 1 AND 5),
    CHECK (cleanliness BETWEEN 1 AND 5),
    CHECK (food BETWEEN 1 AND 5),
    CHECK (overall BETWEEN 1 AND 5),
    CONSTRAINT fk_patient_rating
        FOREIGN KEY (patient_id) REFERENCES Patient(amka)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_rating_hosp
        FOREIGN KEY (hosp_id) REFERENCES Hospitalization(hosp_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
CREATE TABLE Prescription (
    doctor_id BIGINT NOT NULL,
    hosp_id INT NOT NULL,
    drug_id INT NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE,
    dosage VARCHAR(100) NOT NULL,
    frequency VARCHAR(100) NOT NULL,
    PRIMARY KEY (doctor_id, hosp_id, drug_id, start_date),
    CONSTRAINT fk_presc_doctor
        FOREIGN KEY (doctor_id) REFERENCES Doctor(amka)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_presc_hospitalization
        FOREIGN KEY (hosp_id) REFERENCES Hospitalization(hosp_id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_presc_drug
        FOREIGN KEY (drug_id) REFERENCES Drug(drug_id)
        ON DELETE CASCADE ON UPDATE CASCADE
);
CREATE TABLE MedicalAct(
    code VARCHAR(16) NOT NULL,
    name TEXT NOT NULL,
    PRIMARY KEY (code)
);
CREATE TABLE Room(
    room_id INT NOT NULL,
    type VARCHAR(32) NOT NULL,
    PRIMARY KEY (room_id),
    CHECK (type in ('SURGERY', 'INTERVENTION_ROOM'))
);
CREATE TABLE Examination (

```

```

exam_id INT AUTO_INCREMENT,
code VARCHAR(16) NOT NULL,
type VARCHAR(100) NOT NULL,
exam_date DATE NOT NULL,
result TEXT NOT NULL,
cost DECIMAL(10,2) NOT NULL,
hosp_id INT NOT NULL,
doctor_id BIGINT NOT NULL,
PRIMARY KEY (exam_id),
CONSTRAINT fk_exam_hosp
    FOREIGN KEY (hosp_id) REFERENCES Hospitalization(hosp_id)
    ON DELETE CASCADE ON UPDATE CASCADE,
CONSTRAINT fk_exam_doctor
    FOREIGN KEY (doctor_id) REFERENCES Doctor(amka)
    ON DELETE CASCADE ON UPDATE CASCADE,
CONSTRAINT fk_exam_name
    FOREIGN KEY (code) REFERENCES MedicalAct(code)
    ON DELETE CASCADE ON UPDATE CASCADE
);
CREATE TABLE MedicalProcedureOp (
proc_id INT AUTO_INCREMENT,
proc_code VARCHAR(16) NOT NULL,
category VARCHAR(30) NOT NULL,
duration INT NOT NULL,
start_datetime DATETIME NOT NULL,
cost DECIMAL(10,2) NOT NULL,
room_id INT NOT NULL,
hosp_id INT NOT NULL,
PRIMARY KEY (proc_id),
CHECK (category IN ('SURGICAL', 'DIAGNOSTIC', 'THERAPEUTIC')),
CHECK (cost >= 0),
CONSTRAINT fk_proc_hosp
    FOREIGN KEY (hosp_id) REFERENCES Hospitalization(hosp_id)
    ON DELETE CASCADE
    ON UPDATE CASCADE,
CONSTRAINT fk_proc_room
    FOREIGN KEY (room_id) REFERENCES Room(room_id)
    ON DELETE CASCADE
    ON UPDATE CASCADE,
CONSTRAINT fk_proc_code
    FOREIGN KEY (proc_code) REFERENCES MedicalAct(code)
    ON DELETE CASCADE
    ON UPDATE CASCADE
);
CREATE TABLE ProcedureParticipation (
proc_id INT NOT NULL,
amka BIGINT NOT NULL,
role VARCHAR(30) NOT NULL,
PRIMARY KEY (proc_id, amka),
CHECK (role IN ('MAIN_SURGEON', 'ASSISTANT')),
CONSTRAINT fk_part_proc
    FOREIGN KEY (proc_id) REFERENCES MedicalProcedureOp(proc_id)
    ON DELETE CASCADE
    ON UPDATE CASCADE,
CONSTRAINT fk_part_staff
    FOREIGN KEY (amka) REFERENCES Staff(amka)
    ON DELETE CASCADE
    ON UPDATE CASCADE
);
CREATE TABLE Shift (
shift_id INT AUTO_INCREMENT,
date DATE NOT NULL,
type VARCHAR(20) NOT NULL,
dept INT NOT NULL,
PRIMARY KEY (shift_id),
CHECK (type IN ('MORNING', 'AFTERNOON', 'NIGHT')),
CONSTRAINT fk_shift_dept
    FOREIGN KEY (dept) REFERENCES Department(dept_id)

```

```
        ON DELETE CASCADE
        ON UPDATE CASCADE
    );
CREATE TABLE ShiftAssignment (
    amka BIGINT NOT NULL,
    shift_id INT NOT NULL,
    role VARCHAR(30) NOT NULL,
    PRIMARY KEY (amka, shift_id),
    CHECK (role IN ('DOCTOR', 'NURSE', 'ADMIN')),
    CONSTRAINT fk_sa_staff
        FOREIGN KEY (amka) REFERENCES Staff(amka)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_sa_shift
        FOREIGN KEY (shift_id) REFERENCES Shift(shift_id)
        ON DELETE CASCADE        ON UPDATE CASCADE
);
```